

README: Project 3 – STCP: Reliable Transport Layer

Student ID: 2021-18641

Name: Hadong Lee

1. Implementation Strategy

This implementation fully supports unreliable networks (packet loss, reordering, duplication) by explicitly designing buffer management and retransmission logic to tolerate such conditions.

STCP (Simple TCP) is implemented between the mysocket API and an unreliable network layer, providing key TCP-like features:

- Reliable, in-order data delivery
- Retransmission with dynamic timeout (Karn-Partridge RTO)
- Flow and connection management
- Robust segment/interval handling in the receive buffer

1.1 Core Data Structures

- **Connection Context** (`context_t`):
Holds all per-connection state: send/receive windows, current state (FSM), sequence numbers, retransmission tracking, timers (for RTO), and buffer linked lists for both send and receive.
- **Segment Buffer List** (`segment_list_t/segment_t`):
Used for both send and receive queues. Each node stores a segment's start sequence, length, data, FIN flag, and next pointer.

1.2 Protocol Logic

1.2.1 Connection Establishment (3-Way Handshake)

- **Active Side** (`do_active_handshake`):
Implements client-initiated SYN → SYN-ACK → ACK handshake, with retransmission and timeout.
- **Passive Side** (`do_passive_handshake`):
Listens for SYN, sends SYN-ACK, waits for ACK, with proper state and retransmission logic.

1.2.2 Reliable Data Transfer

- **Send Side:**
Application data is segmented to fit the window and buffered.
Segments are retransmitted on timeout using Go-Back-N.
FIN is handled and tracked as part of the buffer, with full state transitions (e.g., FIN_WAIT_x, LAST_ACK, etc.).
- **Receive Side:**
Uses a robust interval-merge algorithm to insert segments (including overlapping/out-of-

order/duplicate arrivals) into the buffer.

Flushes contiguous, in-order data to the application as soon as available.

1.2.3 Robust Receive Buffering (Interval Merge)

- Incoming segments may partially overlap, be duplicated, or arrive out of order.
- Only unique, non-overlapping data is inserted.
- The buffer remains sorted and ready for flush once all bytes up to the next missing sequence are present.

1.2.4 ACK and Retransmission Logic

- Every valid data/FIN segment is ACKed.
- Timeouts trigger retransmission of all un-ACKed segments from `send_base` (Go-Back-N style).
- RTO is dynamically tuned using the Karn-Partridge algorithm based on RTT observations.

1.2.5 Connection Termination

- Supports TCP-style graceful close (handling all relevant FSM states).
 - Ensures all data and FINs are reliably delivered and acknowledged before full teardown.
-

1.3 Notable Functions

(A) Buffer Management

- `insert_segment_to_receive_buffer_robust`
 - Core of robust receive-side buffering: inserts a segment into the receive buffer using an interval-merge algorithm.
 - Handles partial/complete overlap, duplication, and ensures only unique data is stored.
 - Maintains the buffer as a sorted, non-overlapping list, ready for in-order delivery.
- `flush_in_order_segments_from_receive_buffer`
 - Delivers segments to the application whenever a contiguous in-order sequence is available.
 - Removes and frees segments as they are delivered.
 - Handles FIN by signaling end-of-stream and updating state.
- `enqueue_send_buffer` / `dequeue_send_buffer` / `resend_send_buffer`
 - `enqueue_send_buffer`: Adds a new segment to the end of the send buffer as data is sent.
 - `dequeue_send_buffer`: Removes acknowledged segments from the head of the send buffer based on cumulative ACK.
 - `resend_send_buffer`: Retransmits all segments from `send_base` forward (Go-Back-N), used after timeout.

(B) Handshake and State Management

- `do_active_handshake` / `do_passive_handshake`

- Implements the TCP three-way handshake for connection setup.
- Handles retransmission and timer logic for lost handshake messages.
- Initializes all sequence numbers and context.
- **transition_state**
 - Implements the connection state machine (FSM).
 - Cleanly manages state changes for all TCP close/FIN/ACK paths.

(C) ACK, Retransmission, and Timer Logic

- **update_rto**
 - Implements the Karn-Partridge RTT/RTO estimation.
 - Ensures retransmission timeouts are properly adaptive and robust to network conditions.
- **handle_timeout**
 - Called when a timeout occurs: triggers retransmissions, doubles RTO as needed, and increments retransmission counters.
 - Handles connection abortion if the max retransmit count is reached.
- **handle_ack_from_peer**
 - Processes incoming ACKs, advances **send_base**, triggers any pending FIN actions, and resets retransmission state/timers as needed.
- **handle_incoming_segment**
 - Processes incoming data segments: handles window/sequence checks, inserts into receive buffer using robust merge, sends ACK, and flushes ready data to the application.

(D) Debugging / Utilities

- **print_receive_buffer**
 - Utility function for debugging: prints the current state of the receive buffer, including sequence numbers, data ranges, and FIN markers.
- (Other utility functions for segment list initialization, clearing, and header construction are omitted from this summary but exist for modularity.)

2. Program Test

Testing was performed as follows:

1. Correctness & Robustness (Reliable Mode)

- Verified file transfers (up to 10MB files)
- Confirmed correct in-order output even with out-of-order or duplicate arrivals.
- Tested various connection close scenario - FIN sent / FIN received / Timeout

2. Unreliable Network Simulation

- Ran with simulated packet loss, reordering, and duplication enabled.
- Confirmed all data delivered in order, no data loss or corruption, all state transitions correct.

3. Stress/Edge Testing

- Sent fragmented and overlapping segments intentionally (in transport layer).
- Inspected buffer state with debug print functions to ensure proper merge and delivery.

3. Known Bugs / Weaknesses

- No known bugs as of submission.
- Large file transfers are **very slow** due to the small window size (3072 bytes), which significantly limits throughput versus real TCP.
- There is **no idle timeout**: only ACK-based retransmission is implemented. If the peer disconnects ungracefully, the connection may hang rather than closing itself.
- On the **send side**, if the application or network induces severe fragmentation and overlapping retransmissions, the send buffer may temporarily contain segments that slightly overlap with already-ACKed **send_base** data.

This is slightly inefficient (keeps data longer than needed) but does **not** impact correctness, since the assignment's network layer does not model true IP fragmentation or data corruption.

This tradeoff is not considered a bug for the purposes of this project.